Contents

The Threshold S

SC2001 Project 1 — Integration of Merge Sort and Insertion Sort

Merge sort that switches to insertion sort on small subarrays is faster than merge sort. It is also worse, by every measure the assignment asks us to plot. This report reconciles those two facts.

Measured on Python 3.13, Apple Silicon, macOS 24.6, with n up to 10^7. Every figure is generated from the CSVs in `results/`.

Key comparisons	The hybrid never needs fewer than plain merge sort, for any S. Proven exactly.
CPU time	11.9% faster at $n = 10^7$ with the optimal S: 16.56 s against 18.81 s.
Optimal leaf size	7–10, constant across four orders of magnitude of n.

---

1. Implementation (part a)

The hybrid is merge sort with one extra line at the top of the recursion. When a subarray reaches length S or shorter, it goes to insertion sort instead of being split further. The test is applied on every branch independently, so different parts of the tree bottom out at whatever depth their own length reaches.

Three sorts share one counting convention, and each returns the sorted list together with the number of key comparisons performed:

Function	Role
`hybrid(arr, S)`	The hybrid. $S = 1$ degenerates to plain merge sort, since a one-element subarray costs insertion sort nothing.
`mergesort(arr)`	Textbook merge sort, recursing to single elements. The baseline for part (d).
`insertsort(arr)`	Insertion sort with an early exit once the key is in place, so it is adaptive rather than always worst-case.

What counts as a key comparison

A key comparison is one comparison between two array *elements*: `a[j] > key` in insertion sort and `a[i] <= b[j]` in the merge step. Loop bounds such as `j >= 0` and `i < len(a)` are not counted, and neither is the `len(arr) <= S` size test, because none of them inspects a key. This matters — the size test fires once per recursive call and would otherwise inflate small-S counts by a term proportional to n.

Verification

Three checks run whenever the module is executed:

- All three sorts reproduce `sorted()` for n from 0 to 4096, and the hybrid does so for every $S \in \{1, 2, 5, 16, 64, n + 5\}$.
- **`hybrid(arr, 1)` reports exactly the same comparison count as `mergesort(arr)`** — 55,178 against 55,178 at $n = 5000$. Equality to the digit says the switching logic neither double-counts nor skips a single comparison.
- Insertion sort is adaptive: 999 comparisons on a sorted array of 1000 (the $n - 1$ best case) and 499,500 on a reversed one (the $n(n-1)/2$ worst case).

> **Measurement hazard.** Carrying the counter costs about 15% of the runtime, because every level of the recursion packs and unpacks a tuple. Every CPU time in this report therefore comes from a separate set of uncounted twins (`hybrid_nc`, `mergesort_nc`, `insertsort_nc`) that are otherwise line-for-line identical. Comparison counts and timings are never taken from the same run.

---

2. Input data (part b)

Arrays hold uniform random integers in $[1, x]$ with $x = 10^7$. Thirteen sizes span the required range in a 1–2–5 progression — 1000, 2000, 5000, 10,000, …, 10,000,000 — which spaces them evenly on the logarithmic axes part (c) needs.

Arrays are **regenerated from a seed rather than stored**. A 10-million element array occupies roughly 400 MB in memory; as a seed it occupies one integer, and a run stays exactly reproducible. Generation uses numpy when available (about 3× faster) and falls back to the standard library otherwise.

> **Sampling note.** For a fixed seed the smaller arrays are prefixes of the larger ones. This is deliberate: it isolates the effect of n in part (c)(i) rather than mixing it with sampling noise between sizes. Averaging over repetitions requires varying the seed, not just re-running.

---

3. Theoretical analysis

Every count below is **worst case**. That is what admits a closed form, and it is what the Θ classification rests on. Average-case behaviour is reported from measurement instead, in sections 4 and 5.

3.1 Insertion sort

Inserting $a[i]$ into the sorted prefix of length i compares the key against prefix elements from the right until one is smaller. That is at most i comparisons, and at least one when the key is already in place. Summing over $i = 1, \dots, m-1$:

$$I_{\text{worst}}(m) \;=\; \sum_{i=1}^{m-1} i \;=\; \frac{m(m-1)}{2}, \qquad\qquad I_{\text{best}}(m) \;=\; m-1$$

The early exit is what separates the two. Without it every insertion would scan its whole prefix and even a sorted array would cost $m(m-1)/2$.

3.2 Merge sort

Merging sorted runs of lengths a and b removes one element per comparison, and the final element is copied out with no comparison at all, so a merge costs at most $a + b - 1$. A subarray of length m therefore satisfies, with $MS(1) = 0$,

$$MS(m) \;=\; MS\big(\lfloor \tfrac{m}{2} \rfloor\big) \;+\; MS\big(\lceil \tfrac{m}{2} \rceil\big) \;+\; (m-1)$$

For m a power of two this solves level by level. Level k holds $m/2^k$ merges of two runs of length 2^{k-1}, each costing at most $2^k - 1$, so the level costs $m - m/2^k$. Summing over $k = 1, \dots, \log_2 m$:

$$MS(m) \;=\; \sum_{k=1}^{\log_2 m} \Big(m - \frac{m}{2^k}\Big) \;=\; m \log_2 m \;-\; m \;+\; 1$$

3.3 The hybrid

Take n and S both powers of two, so all n/S leaves have length exactly S. Each leaf costs at most $S(S-1)/2$, and the $\log_2(n/S)$ merge levels above them cost $n \log_2(n/S) - n/S + 1$ by the same sum as in 3.2. Hence

$$\boxed{C(n,S) \;=\; \frac{n(S-1)}{2} \;+\; n \log_2 \frac{n}{S} \;-\; \frac{n}{S} \;+\; 1}$$

Setting $S = 1$ recovers $MS(n) = n \log_2 n - n + 1$, as it must. Subtracting the two gives the price of the threshold — linear in n, with a coefficient depending only on S:

$$\Delta(S) \;=\; C(n,S) - C(n,1) \;=\; n\left[\frac{S-1}{2} \;-\; \log_2 S \;+\; 1 \;-\; \frac{1}{S}\right]$$

S	1	2	4	8	16	32	64
$\Delta(S)/n$	0	0	0.250	1.375	4.438	11.469	26.484

Table 1. The worst-case cost of the threshold, per element.

$\Delta(S)$ is zero at $S = 1$ and $S = 2$ and strictly increasing thereafter. **No threshold above the smallest ever reduces the worst-case comparison count** — the hybrid can only tie plain merge sort, never beat it. Sections 5 and 8 confirm this holds for the measured average case too.

The intuition is visible in the formula: raising S trades $\log_2 S$ merge levels, each costing about n, for leaves that cost about $nS/4$. The first term saves logarithmically and the second spends linearly, so the exchange is unfavourable from the start.

3.4 Asymptotics

For any *fixed* S the leaf term is $\Theta(nS) = \Theta(n)$, which the $n \log n$ term dominates:

$$C(n, S) \;=\; n \log_2 n \;+\; n\left[\frac{S-1}{2} - \log_2 S\right] \;-\; \frac{n}{S} \;+\; 1 \;=\; \Theta(n \log n)$$

The threshold changes the constant, never the complexity class. This only breaks if S is allowed to grow with n: at $S = \Theta(n)$ the whole array goes to insertion sort and the cost becomes $\Theta(n^2)$.

3.5 What happens at one subarray

The worst-case formula says the threshold costs comparisons. Measurement says the same for the average case. Over 200,000 random arrays per size:

m	Insertion, measured avg	Merge, measured avg	Cheaper
2	1.00	1.00	tie
3	2.66	2.67	insertion
4	4.92	4.67	merge
6	11.06	9.83	merge
8	19.28	15.74	merge
16	72.46	45.70	merge
32	275.39	121.49	merge
64	1067.44	305.17	merge

Table 2. Average key comparisons for one subarray, measured.

Insertion sort ties at $m = 2$, wins at $m = 3$ by 0.01 comparisons, and loses at every larger size with the gap widening fast. This is the average-case counterpart of Table 1 and it points the same way: **on key comparisons, cutting the recursion short is never an improvement.** Whatever the hybrid is buying, it is not fewer comparisons — section 7 identifies what it is.

4. Comparisons against n (part c(i))

With $S = 8$ held constant, both sorts were run on all thirteen sizes. Plotting $C(n)$ directly against n on log–log axes is a poor test, because almost any superlinear curve looks straight there. Plotting $C(n)/n$ against $\log n$ is sharper: the theory says this quantity is *linear*, so anything other than a straight line falsifies the model.

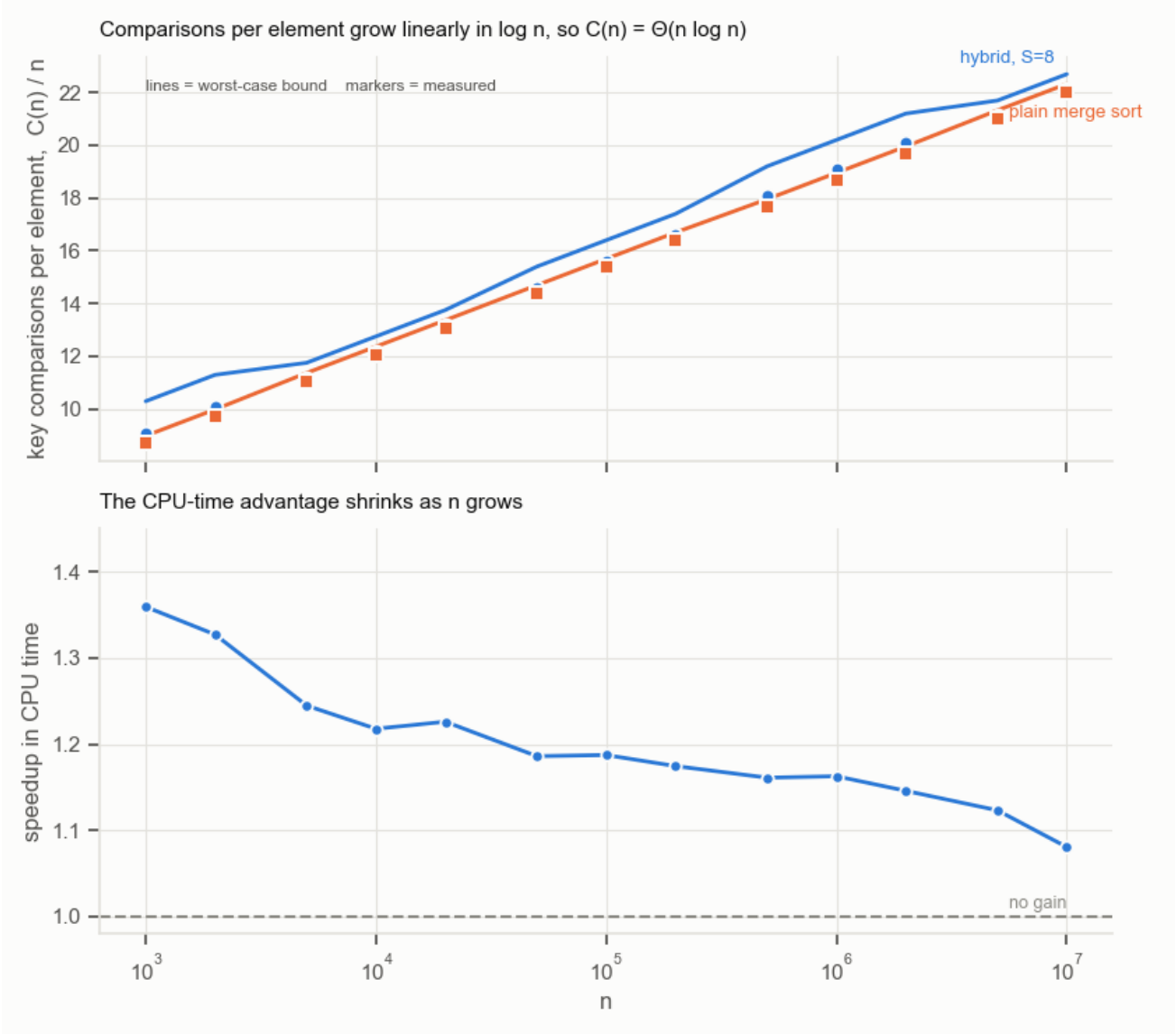

Figure 1. Top: comparisons per element against n. Markers are measured, lines are the worst-case bound $C(n, S)/n$ of section 3.3 evaluated over the actual splits. Measurement sits just below the bound and both rise linearly in $\log n$, which is what $\Theta(n \log n)$ predicts. Bottom: the CPU-time advantage, decaying from 1.36× to 1.08× as n grows.

n	Hybrid comparisons	Merge comparisons	Excess	Hybrid CPU	Merge CPU	Speedup
1,000	9,088	8,717	+4.26%	0.00053 s	0.00072 s	1.36×
2,000	20,217	19,422	+4.09%	0.00121 s	0.00161 s	1.33×
5,000	55,766	55,237	+0.96%	0.00373 s	0.00464 s	1.24×
10,000	121,468	120,431	+0.86%	0.00825 s	0.01005 s	1.22×
20,000	263,085	260,942	+0.82%	0.0189 s	0.0232 s	1.23×
50,000	729,394	718,417	+1.53%	0.0501 s	0.0595 s	1.19×
100,000	1,558,492	1,536,560	+1.43%	0.1071 s	0.1272 s	1.19×
200,000	3,315,992	3,272,943	+1.32%	0.2304 s	0.2706 s	1.17×
500,000	9,036,918	8,837,196	+2.26%	0.6255 s	0.7263 s	1.16×
1,000,000	19,073,414	18,674,561	+2.14%	1.3307 s	1.5472 s	1.16×
2,000,000	40,148,187	39,349,331	+2.03%	2.8853 s	3.3060 s	1.15×
5,000,000	105,556,634	105,052,333	+0.48%	7.9996 s	8.9849 s	1.12×

n	Hybrid comparisons	Merge comparisons	Excess	Hybrid CPU	Merge CPU	Speedup
10,000,000	221,112,527	220,105,208	+0.46%	17.576 s	18.994 s	1.08×

Table 3. Part (c)(i), $S = 8$. Comparison counts are exact; CPU times are medians of 7 runs at the small sizes falling to 2 at 10 million.

The excess column does not fall smoothly, and the reason recurs throughout part (c). At $S = 8$ the leaves are not all of length 8 — they are whatever $\lceil n/2^k \rceil$ lands at or below 8. At $n = 10^6$ that is 7–8, but at $n = 10^7$ it is 4–5, much closer to plain merge sort, which is why the excess there is only 0.46%. **The same S means different things at different n.**

Figure 1 answers "compare with your theoretical analysis". Measured counts sit just under the worst-case bound of section 3.3 — 1.4% to 2.9% below it for plain merge sort, 2.6% to 5.9% for the hybrid. The hybrid's gap is the wider one because its leaf term charges insertion sort at $S(S-1)/2$, the worst case, while random data costs roughly half that. Both curves are straight in $\log n$ with the slope $C(n, S)/n \sim \log_2 n$ demands, which is the $\Theta(n \log n)$ claim of section 3.4 confirmed over four orders of magnitude.

---

5. Comparisons against S (part c(ii))

Twenty-two values of S at $n = 1{,}000{,}000$, each timed five times. Comparison counts are deterministic given the array and S, so those were measured once; only the timings needed repetition.

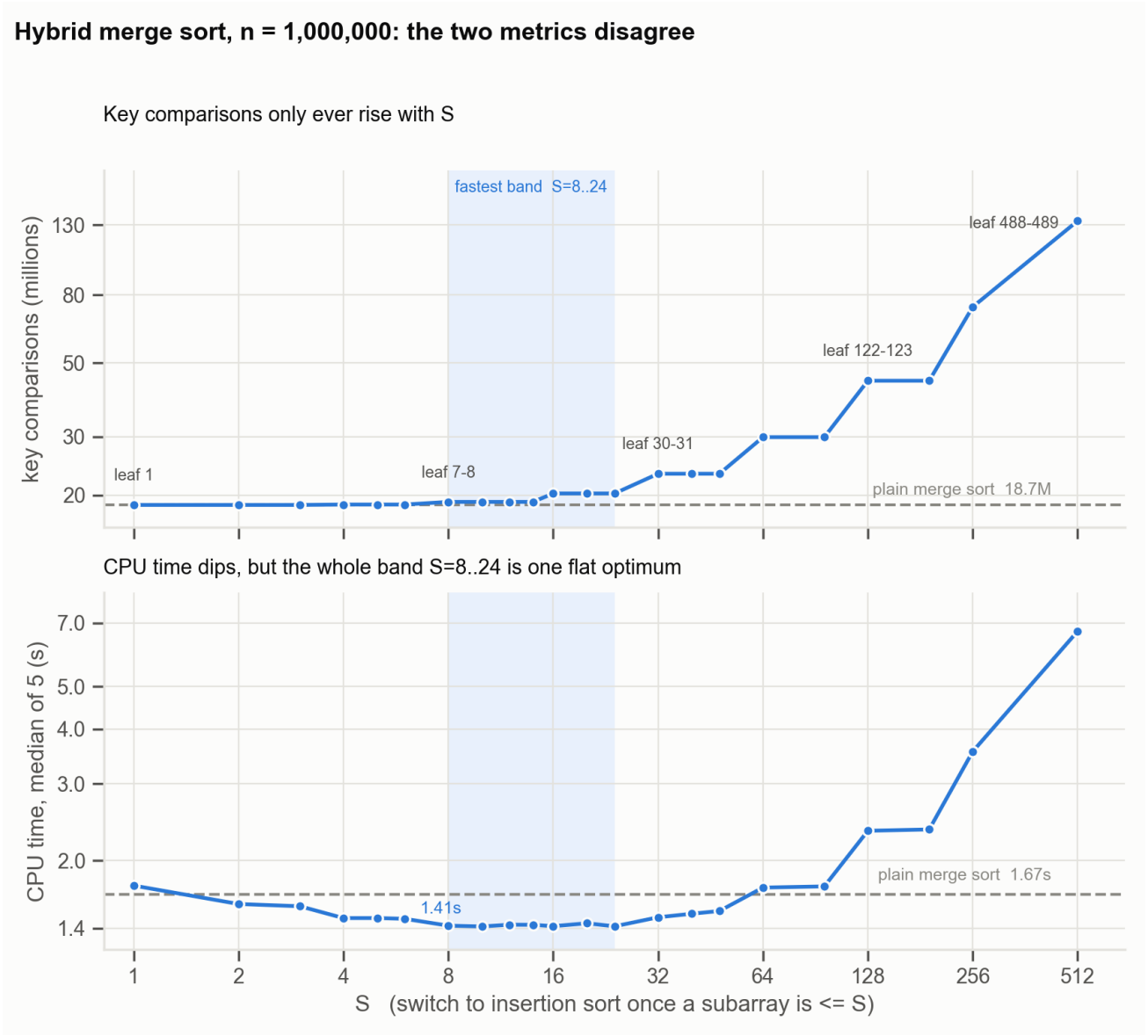

Figure 2. The two metrics point in opposite directions. Key comparisons (top) rise monotonically and never drop below the plain merge sort line. CPU time (bottom) falls to a flat minimum around $S = 8$–24 and only then rises. A twin-axis version of this chart would invite reading a crossover that is purely an artefact of where the two scales were pinned.

S is quantised

The comparison counts come in plateaus, and the plateau boundaries align exactly with the halving ladder $\lceil n/2^k \rceil$. Every S inside one plateau produces the identical execution — the counts are equal to the digit, not merely close. $S = 8, 10, 12, 14$ **are not four nearby choices; they are one algorithm measured four times**, and the ~1% spread in their timings is measurement noise, nothing else.

S	Leaf sizes	Key comparisons	vs merge	CPU median	Speedup
1	1	18,674,561	1.00×	1.7526 s	0.95×
2	1–2	18,674,561	1.00×	1.5903 s	1.05×
3	2–3	18,674,513	1.00×	1.5717 s	1.06×

S	Leaf sizes	Key comparisons	vs merge	CPU median	Speedup
4	3–4	18,727,864	1.00×	1.4766 s	1.13×
5	3–4	18,727,864	1.00×	1.4769 s	1.13×
6	3–4	18,727,864	1.00×	1.4710 s	1.14×
8	**7–8**	**19,073,414**	**1.02×**	**1.4188 s**	**1.18×**
10	**7–8**	**19,073,414**	**1.02×**	**1.4128 s**	**1.18×**
12	**7–8**	**19,073,414**	**1.02×**	**1.4262 s**	**1.17×**
14	**7–8**	**19,073,414**	**1.02×**	**1.4252 s**	**1.17×**
16	**15–16**	**20,222,242**	**1.08×**	**1.4150 s**	**1.18×**
20	**15–16**	**20,222,242**	**1.08×**	**1.4370 s**	**1.16×**
24	**15–16**	**20,222,242**	**1.08×**	**1.4138 s**	**1.18×**
32	30–31	23,186,948	1.24×	1.4829 s	1.13×
40	30–31	23,186,948	1.24×	1.5108 s	1.11×
48	30–31	23,186,948	1.24×	1.5328 s	1.09×
64	61–62	29,880,484	1.60×	1.7342 s	0.96×
96	61–62	29,880,484	1.60×	1.7451 s	0.96×
128	122–123	44,185,373	2.37×	2.3394 s	0.71×
192	122–123	44,185,373	2.37×	2.3558 s	0.71×
256	244–245	73,735,430	3.95×	3.5532 s	0.47×
512	488–489	133,829,093	7.17×	6.6962 s	0.25×

Table 4. Part (c)(ii), $n = 1{,}000{,}000$. The **leaf sizes** column gives the subarray lengths insertion sort actually receives, and explains every plateau. Bold rows are within 1% of the fastest. Plain merge sort: 18,674,561 comparisons, 1.6713 s.

Comparing against theory: every measured count lies below the worst-case bound of section 3.3, and the two track closely over the range of S that matters — 1.5% apart at $S \leq 3$, 5.6% at $S = 8$, 12.3% at $S = 16$. The gap then widens to 47% by $S = 512$, because the bound charges the leaves at insertion sort's worst case $S(S-1)/2$ while random data costs about half of it, and at large S that leaf term is nearly the whole cost. **The bound is tight where the algorithm is useful and loose where it is not.**

> The comparison curve has its minimum at $S = 3$, where it undercuts plain merge sort by 48 comparisons out of 18.7 million — a 0.0003% improvement, which is the near-tie at $m = 3$ in Table 2 showing up at full scale. **If part (c)(ii) is read strictly as an optimisation over key comparisons, the answer is $S = 3$ and the hybrid is pointless.** The CPU-time curve is where the algorithm justifies itself, and section 7 explains why the two disagree.

---

6. Choosing S (part c(iii))

The sweep was repeated at four sizes spanning four orders of magnitude, timing each S against plain merge sort on the same array.

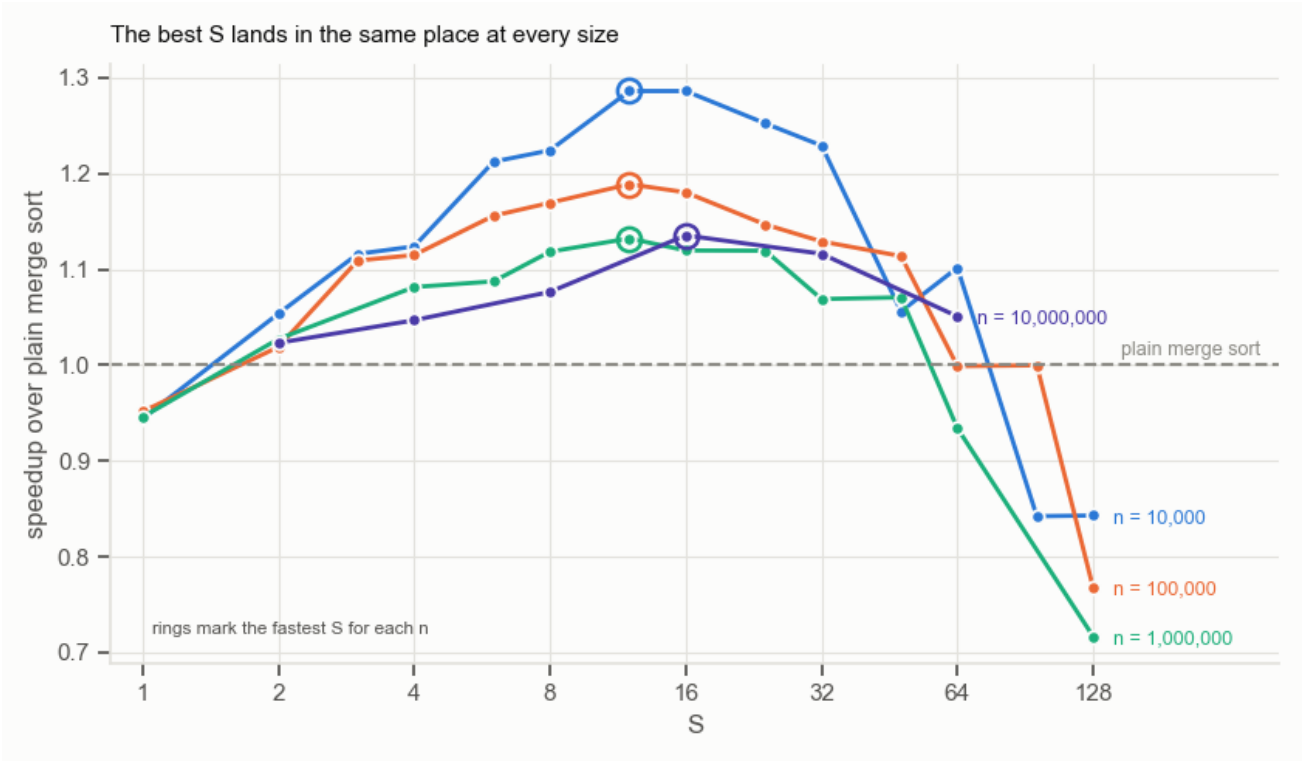

Figure 3. Speedup over plain merge sort against S, one curve per input size; rings mark each curve's fastest point. The optima sit on top of one another despite the sizes differing by 1000×. The $n = 10^7$ curve uses a coarser grid because each point costs about 35 seconds.

n	Best S	Leaf sizes	Hybrid CPU	Merge CPU	Speedup
10,000	12	9–10	0.0081 s	0.0104 s	1.29×
100,000	12	6, 7, 12	0.1063 s	0.1263 s	1.19×
1,000,000	12	7–8	1.3484 s	1.5267 s	1.13×
10,000,000	16	9–10	16.5645 s	18.8082 s	1.14×

Table 5. Fastest S at each size, and the leaf sizes it corresponds to. The optimal S shifts; the optimal leaf size does not.

> **The optimal leaf size is 7–10 at every size tested**, even though the S producing it moves from 12 to 16. This is what the theory predicts: the choice at a subarray of length m weighs the cost of insertion-sorting it against the cost of splitting it further, plus the per-call overhead, and *none of those quantities mentions* n. The decision is local to the subarray, so its answer cannot depend on how large the whole array is.

The practical recommendation follows: **report the optimal leaf size, not the optimal** S**.** Quoting "$S = 12$" is an artefact of which n was tested — at $n = 10^7$ that same 12 gives leaves of 4–5 and costs 5% of the available speedup. Quoting "cut at leaf size ≈ 8" transfers.

One further caveat on precision: at $n = 10^6$ the values $S = 8, 10, 12, 14$ are the same algorithm, and $S = 16, 20, 24$ differ from them by 0.07% in median time over five runs. No amount of extra repetition will separate those two plateaus. The honest answer is an interval, not a point.

---

7. Where the time actually goes

Sections 3 and 5 leave an apparent contradiction: the hybrid does more key comparisons yet finishes sooner. The resolution is that key comparisons are not what merge sort spends its time on.

7.1 What each sort actually executes

Operation	Merge sort	Insertion sort
Recursive calls	15	1
… of which sort a single element	8	—
Merge steps	7	0
List slices	21	0
New lists allocated	15	1
Appends / extends	17 / 7	0
Key comparisons	**~16**	**~19**

Table 6. Operations performed by one sort of an array of 8 elements.

Eight of merge sort's fifteen calls sort a one-element array. They check a length, copy a one-element list, and return. They perform zero comparisons and move zero elements; they exist only because the recursion needs a base case.

7.2 Attributing the time

Stripping merge sort down one layer at a time isolates each cost. Microseconds per sort of an 8-element array, best of 7 passes over 4000 arrays:

Layer	μs / sort	Added
Recursion + 21 slices + 8 leaf copies only	1.046	—
… plus allocating the 7 result lists	1.340	+0.294
… plus the merging itself — **full merge sort**	**2.810**	+1.470
Insertion sort, complete	**0.980**	
Saved by switching at $m = 8$	**1.830**	

Table 7. Merge sort's cost, decomposed. The scaffolding is half the total.

The 1.830 μs splits cleanly in two:

- **1.340 μs — the entire recursion tree disappears.** Fifteen calls collapse to one; 21 slices, 8 leaf copies and 7 result lists collapse to a single `list(arr)`. This is 47.7% of merge sort's total cost, spent taking the array apart and putting it back together.
- **0.490 μs — merge's inner loop is dearer per element.** Merge's merging step alone costs 1.470 μs, which is **1.50× the cost of the whole insertion sort** despite performing three fewer comparisons. Placing an element costs `new.append(x)`, an attribute lookup and a method call, where insertion sort does `arr[j+1] = arr[j]`, a store. The merge loop also re-evaluates `len(a)` and `len(b)` on every iteration.

7.3 Why the tradeoff is so lopsided

The two costs are distributed differently across the recursion tree, and that asymmetry is the whole explanation.

Every level of the tree merges n elements in total, so **each level costs about n comparisons**. With $\log_2 n$ levels, the comparison work is spread evenly: at $n = 10^6$ each of the ~20 levels holds about 5% of the total.

Node counts are not spread evenly. A tree over n elements has n leaves and $n-1$ internal nodes, so **the bottom level alone holds half of all nodes**:

$$\frac{\text{nodes in the bottom } j \text{ levels}}{\text{total nodes}} = \frac{n + \frac{n}{2} + \cdots + \frac{n}{2^{j-1}}}{2n-1} \approx 1 - 2^{-j}$$

so the bottom three levels hold 87.5% of them.

> Cutting at leaf size 8 removes the bottom three levels. That discards **87.5% of all function calls, slices and allocations** in exchange for the comparison work of three levels — which insertion sort then redoes at a slightly worse rate, costing about 2% more comparisons. Measured at $n = 10^6$: **+2.1% comparisons, −15% CPU time.**

7.4 The two crossovers

Because the metrics measure different things, they cross at different subarray sizes. Measured over 200,000 random arrays per size for comparisons, and best-of-5 timings:

m	Cheaper on comparisons	Faster in CPU time	Merge / insertion time
2	tie	insertion	2.32×
4	merge	insertion	2.99×
8	merge	insertion	2.77×
16	merge	insertion	2.30×
32	merge	insertion	1.56×
64	merge	**merge**	0.87×

Table 8. Which sort wins at a small subarray, by each metric. The comparison crossover is at $m \approx 3$; the CPU crossover is at $m \approx 48$.

The assignment's own framing anticipates this: it motivates the hybrid by "the overhead of many recursive calls", not by comparison counts. Table 7 is that sentence measured.

---

8. Against the original merge sort (part d)

At $n = 10{,}000{,}000$, using the optimal $S = 16$ from part (c)(iii), against the textbook merge sort of section 1. Both sorts were checked to produce identical output.

	Key comparisons	vs merge	CPU time	vs merge
Hybrid, $S = 16$	**226,423,622**	**+2.87%**	**16.564 s**	**−11.9%**
Hybrid, $S = 8$	221,112,527	+0.46%	17.472 s	−7.1%
Plain merge sort	*220,105,208*	—	*18.808 s*	—

Table 9. Head to head at 10 million integers. Both algorithms sorted the same array.

The result is the report's thesis in one row. **The hybrid performs 6.3 million more key comparisons and still finishes 2.24 seconds sooner.** On the metric parts (c)(i) and (c)(ii) ask us to plot, it loses; on the metric part (d) asks us to measure, it wins by 11.9%.

The $S = 8$ row shows the quantisation effect from another angle: it costs far fewer extra comparisons (+0.46%) precisely because its leaves are only 4–5 elements, and it is correspondingly slower, capturing 7.1% instead of 11.9%. Fewer comparisons, less speed — the two metrics are anticorrelated across the whole useful range of S.

> **Run-to-run variance.** Plain merge sort at 10 million measured 18.808 s in the part (c)(iii) run and 18.298 s in a separate part (d) run, a 2.7% spread between sessions. Differences below roughly 3% at this size should not be treated as real. The 11.9% gap is comfortably outside that band; the gap between $S = 12$ and $S = 16$ is not.

---

9. Threats to validity

The optimum is an implementation constant, not an algorithm property. Table 7 attributes 47.7% of merge sort's cost to slicing and allocation. An index-based in-place merge sort, using one preallocated buffer and passing (lo, hi) instead of sublists, removes most of that, leaving only the function calls. Its crossover would sit at a smaller m and its optimal leaf size below 8. The figure "leaf size ≈ 8" is specific to this implementation and this language; the *reasoning* transfers, the number does not. This was not measured and is stated as expectation, not result.

Only uniform random data. Insertion sort's early exit makes it strongly adaptive — 999 comparisons on sorted input against 499,500 on reversed. On partially ordered data the optimal S would be larger, possibly much larger. Nothing here measures that regime.

Nested datasets. With a fixed seed, each size is a prefix of the next. This sharpens part (c)(i) by removing between-size sampling noise, but the thirteen points are not independent draws, so confidence intervals across sizes would be optimistic.

Timing repetitions thin at the top. 10-million-element runs were repeated twice, 5-million three times. Combined with the 2.7% between-session variance, the large-n speedups carry roughly $\pm 3\%$ uncertainty. The comparison counts carry none — they are exact.

Duplicate keys. With values from $[1, 10^7]$ and $n = 10^7$, about 37% of entries are duplicates. The merge uses `a[i] <= b[j]`, so ties resolve to the left run, making the sort stable and fixing the tie cost at one comparison. A different tie convention would shift counts slightly.

---

10. Conclusion

The hybrid is worth using, and not for the reason the metric in parts (c)(i) and (c)(ii) would suggest.

1. **On key comparisons the hybrid never wins.** In the worst case $\Delta(S) = 0$ only at $S = 1, 2$ and grows from there; in the measured average case the best threshold is $S = 3$, saving 48 comparisons out of 18.7 million. Both say the same thing.
2. **On CPU time it wins by 11.9% at 10 million.** The gain comes from deleting the bottom three levels of the recursion tree, which hold 87.5% of the calls, slices and allocations but only about 15% of the comparison work.
3. **The optimal leaf size is 7–10 and does not depend on n**, because the decision at a subarray involves only that subarray's length. Report the leaf size; the S achieving it is quantised by $\lceil n/2^k \rceil$ and moves with the input size.

4. **For any fixed S the hybrid remains $\Theta(n \log n)$.** The threshold buys a constant factor, never a complexity class.

The broader methodological point is that the two metrics are anticorrelated over the entire useful range of S, so a report that optimises key comparisons and a report that optimises runtime reach opposite conclusions from the same data. Which is right depends on what the count was ever a proxy for — and in pure Python, on an 8-element subarray, it is a proxy for about half of the work.

Appendix — reproducing this

File	Contents
`hybrid.py`	The three sorts with comparison counting, plus the uncounted twins used for timing. Running it executes the verification suite.
`datagen.py`	Seeded array generation and the size ladder.
`experiments.py`	Parts (c)(i), (c)(iii) and (d). About 15 minutes; writes three CSVs.
`bench.py`	Part (c)(ii): the S sweep, plus the `leafsizes` helper behind the plateau column.
`overhead.py`	Tables 5 and 6. Must run on an otherwise idle machine.
`plots.py`	Figures 1–3, drawn from the CSVs.

Every number is generated by the scripts above and stored in `results/*.csv`. Comparison counts are exact and reproducible from the seed; CPU times were measured on an idle Apple Silicon machine under Python 3.13 and will differ on other hardware.